

ใบงานที่ 7

Tree

วัตถุประสงค์ ศึกษาและเพิ่มทักษะ การจำลองการทำงานของต้นไม้ ดังนี้

การทดลองที่1 ให้ทดลองเขียนโปรแกรมโดยมีโครงสร้างของโหนดด้านล่างนี้ และทำความเข้าใจตามทฤษฎี

```
typedef struct Node {int Info;  
    struct Node *Left;  
    struct Node *Right;  
    struct Node *Mother;  
}TYPE_NODE;
```

```
typedef struct Node *TYPE_NODEPTR;
```

1. เขียนเมนูสำหรับโปรแกรมจำลองต้นไม้ โดยมีเมนูดังนี้

- | | |
|----------------------|--------------------------|
| 1. Create Root Node | 6. Go to Mother Node |
| 2. Insert Left Node | 7. Display Current Node |
| 3. Insert Right Node | 8. Pre-Order Traversal |
| 4. Go to Left | 9. In-Order Traversal |
| 5. Go to Right | 10. Post-Order Traversal |

2. เขียนฟังก์ชันสร้าง Tree แล้วกำหนดให้ Pointer T ชี้ที่ Root Node เสมอ และทดสอบรันโปรแกรม

3. สร้างฟังก์ชัน DisplayC เพื่อแสดงข้อมูลของโหนดปัจจุบันที่พอยน์เตอร์ชี้อยู่ในต้นไม้

```
TYPE_NODEPTR  
CreateNode(int data) {  
    if (root != NULL) {  
        printf("The tree already  
has a root node!\n");  
        return root;  
    }
```

```
TYPE_NODEPTR newNode =  
(TYPE_NODEPTR)malloc(sizeof  
f(struct TreeNode));  
newNode->data = data;
```

```
void  
Displaytree(TYPE_NODEPTR  
node){  
    if (node != NULL) {  
        Displaytree(node->left);  
        printf(" %d ->", node->  
data);  
        Displaytree(node->  
right);  
    }
```

```
newNode->left = NULL;
newNode->right = NULL;
newNode->Mother = NULL;
```

```
root = newNode;
roottmp = newNode;
printf("Create First Node
Successfully!\n");
return root;
}
```

```
void printMenu(void){
    const char *Select[11] = {
        "Create Root Node",
        "Insert Left Node",
        "Insert Right Node",
        "Go to Left",
        "Go to Right",
        "Go to Mother",
        "Display Tree (from current position)",
        "Pre-Order Traversal (whole tree)",
        "In-Order Traversal (whole tree)",
        "Post-Order Traversal (whole tree)",
        "Exit The Program (Enter 0)"
    };
    printf("\n===== \n");
    printf("|          SELECT FUNCTION          | \n");
    printf("===== \n");
    for (int i = 0; i < 11; i++) {
        printf(" %2d %-45s\n", i + 1, Select[i]);
    }
    if (roottmp != NULL) {
        printf("----- \n");
        printf(" Current position: %d\n", roottmp->data);
    }
    printf("===== \n");
    printf("Enter your choice: ");
}
}
```

การทดลองที่ 2 สร้างฟังก์ชัน InsertLeft และ InsertRight

1. เขียนฟังก์ชัน InsertLeft เพื่อเพิ่มโหนดลูกทางซ้ายของโหนดที่ พอยน์เตอร์ (P) ชื่ออยู่ปัจจุบัน และทดสอบรันโปรแกรม
2. เขียนฟังก์ชัน InsertRight เพื่อเพิ่มโหนดลูกทางขวาของโหนดที่ พอยน์เตอร์ (p) ชื่ออยู่ปัจจุบัน และทดสอบรันโปรแกรม

```
TYPE_NODEPTR
insert_left(TYPE_NODEPTR
t, int data){
    if (t == NULL) {
        printf("!! ERROR: no root
yet, please create the
first node !!\n");
        return NULL;
    }
    if (t->left != NULL) {
        printf("(left child
already exists: %d)\n", t-
>left->data);
        return t;
    }
}
```

```
TYPE_NODEPTR
insert_right(TYPE_NODEPTR
t, int data){
    if (t == NULL) {
        printf("!! ERROR: no root
yet, please create the
first node !!\n");
        return NULL;
    }
    if (t->right != NULL) {
        printf("(right child
already exists: %d)\n", t-
>right->data);
        return t;
    }
}
```

```
TYPE_NODEPTR newNode =  
(TYPE_NODEPTR)malloc(sizeof(struct TreeNode));  
newNode->data = data;  
newNode->left = NULL;  
newNode->right = NULL;  
newNode->Mother = t;
```

```
t->left = newNode;  
roottmp = newNode; //  
cursor เลื่อนตาม node ที่เพิ่ม  
insert  
return t;  
}
```

```
=====
Enter your choice: 2
Enter data to insert left of 111: 22

=====
|          SELECT FUNCTION          |
=====
1) Create Root Node
2) Insert Left Node
3) Insert Right Node
4) Go to Left
5) Go to Right
6) Go to Mother
7) Display Tree (from current position)
8) Pre-Order Traversal (whole tree)
9) In-Order Traversal (whole tree)
10) Post-Order Traversal (whole tree)
11) Exit The Program (Enter 0)
=====
Current position: 22
=====
Enter your choice: █
```

```
TYPE_NODEPTR newNode =  
(TYPE_NODEPTR)malloc(sizeof(struct TreeNode));  
newNode->data = data;  
newNode->left = NULL;  
newNode->right = NULL;  
newNode->Mother = t;
```

```
t->right = newNode;  
roottmp = newNode; //  
cursor เลื่อนตาม node ที่เพิ่ม  
insert  
return t;  
}
```

```
=====
Enter your choice: 3
Enter data to insert right of 22: 333

=====
|          SELECT FUNCTION          |
=====
1) Create Root Node
2) Insert Left Node
3) Insert Right Node
4) Go to Left
5) Go to Right
6) Go to Mother
7) Display Tree (from current position)
8) Pre-Order Traversal (whole tree)
9) In-Order Traversal (whole tree)
10) Post-Order Traversal (whole tree)
11) Exit The Program (Enter 0)
=====
Current position: 333
=====
Enter your choice: █
```

การทดลองที่ 3 สร้างฟังก์ชันในการท่องเข้าไปในต้นไม้ 3 รูปแบบ

1. PreOrder – แบบก่อนลำดับ (Root – Left – Right)
2. InOrder – แบบตามลำดับ (Left - Root – Right)
3. PostOrder – แบบหลังลำดับ (Left – Right - Root)

```
void  
preOrderTraversal  
(TYPE_NODEPTR  
node) {  
if (node != NULL)  
{  
printf(" %d -> ",  
node->data);  
preOrderTraversal  
(node->left);  
preOrderTraversal  
(node->right);  
}  
}
```

```
void  
inorderTraversal(  
TYPE_NODEPTR  
node) {  
if (node != NULL)  
{  
inorderTraversal(  
node->left);  
printf(" %d -> ",  
node->data);  
inorderTraversal(  
node->right);  
}  
}
```

```
void  
postOrderTraversa  
l(TYPE_NODEPTR  
node) {  
if (node != NULL)  
{  
postOrderTraversa  
l(node->left);  
postOrderTraversa  
l(node->right);  
printf(" %d -> ",  
node->data);  
}  
}
```

การทดลองที่ 4 ทดลองรันโปรแกรมลักษณะตามด้านล่างนี้ แล้วตรวจสอบผลลัพธ์
ด้านขวา

<pre> InsertLeft(P,4); InOrder(T); printf("\n"); InsertRight(P,12); InOrder(T); printf("\n"); P=GotoLeft(P); InsertLeft(P,2); InOrder(T); printf("\n"); InsertRight(P,6); InOrder(T); printf("\n"); P=GotoMother(P); P=GotoRight(P); InsertLeft(P,10); InOrder(T); printf("\n"); InsertRight(P,14); InOrder(T); printf("\n"); </pre>	<pre> ----- 7 ----- 8 4 8 4 8 12 2 4 8 12 2 4 6 8 12 2 4 6 8 10 12 2 4 6 8 10 12 14 </pre>
<pre> #include <stdio.h> #include <stdlib.h> typedef struct TreeNode{ int data; struct TreeNode* left; struct TreeNode* right; struct TreeNode* Mother; }Treenode; typedef struct TreeNode *TYPE_NODEPTR; TYPE_NODEPTR root = NULL; TYPE_NODEPTR roottmp = </pre>	<pre> Enter your choice: 1 Enter data to create first node: 8 Create First Node Successfully! ===== SELECT FUNCTION ===== 0) Exit The Program (Enter 0) 1) Create Root Node 2) Insert Left Node 3) Insert Right Node 4) Go to Left 5) Go to Right 6) Go to Mother 7) Display Tree (from current position) 8) Pre-Order Traversal (whole tree) 9) In-Order Traversal (whole tree) 10) Post-Order Traversal (whole tree) 11) option 11; ----- Current position: 8 ===== Enter your choice: 11 -----7----- 8 4 8 4 8 12 2 4 8 12 2 4 6 8 12 2 4 6 8 10 12 2 4 6 8 10 12 14 ===== </pre>

```
NULL;
```

```
int running = 1;
```

```
TYPE_NODEPTR
```

```
CreateNode(int data) {  
    if (root != NULL) {  
        printf("The tree already  
has a root node!\n");  
        return root;  
    }  
}
```

```
TYPE_NODEPTR newNode =  
(TYPE_NODEPTR)malloc(sizeof(struct TreeNode));  
newNode->data = data;  
newNode->left = NULL;  
newNode->right = NULL;  
newNode->Mother = NULL;
```

```
root = newNode;  
roottmp = newNode;  
printf("Create First  
Node Successfully!\n");  
return root;  
}
```

```
TYPE_NODEPTR  
insert_left(TYPE_NODEPTR  
t, int data){  
    if (t == NULL) {  
        printf("!! ERROR: no  
root yet, please create
```

```
the first node !!\n");
return NULL;
}
if (t->left != NULL) {
printf("(left child
already exists: %d)\n",
t->left->data);
return t;
}
```

```
TYPE_NODEPTR newNode =
(TYPE_NODEPTR)malloc(sizeof(struct TreeNode));
newNode->data = data;
newNode->left = NULL;
newNode->right = NULL;
newNode->Mother = t;
```

```
t->left = newNode;
//roottmp = newNode;
return t;
}
```

```
TYPE_NODEPTR
insert_right(TYPE_NODEPTR
R t, int data){
if (t == NULL) {
printf("!! ERROR: no
root yet, please create
the first node !!\n");
return NULL;
}
if (t->right != NULL) {
```

```
printf("(right child  
already exists: %d)\n",  
t->right->data);  
return t;  
}
```

```
TYPE_NODEPTR newNode =  
(TYPE_NODEPTR)malloc(sizeof(struct TreeNode));  
newNode->data = data;  
newNode->left = NULL;  
newNode->right = NULL;  
newNode->Mother = t;
```

```
t->right = newNode;  
//roottmp = newNode;  
return t;  
}
```

```
void  
Displaytree(TYPE_NODEPTR  
node){  
if (node != NULL) {  
Displaytree(node->left);  
printf(" %d ->", node->data);  
Displaytree(node->right);  
}  
}
```

```
TYPE_NODEPTR  
gotoLeft(TYPE_NODEPTR t)
```

```
{  
if (t->left == NULL) {  
printf("No left child!  
\n");  
return NULL;  
}  
roottmp = t->left;  
//printf(" %d ",  
roottmp->data);  
return roottmp;  
}
```

```
TYPE_NODEPTR  
gotoRight(TYPE_NODEPTR  
t){  
if (t->right == NULL) {  
printf("No right child!  
\n");  
return NULL;  
}  
roottmp = t->right;  
//printf(" %d ",  
roottmp->data);  
return roottmp;  
}
```

```
TYPE_NODEPTR  
gotoMother(TYPE_NODEPTR  
t){  
if (t->Mother == NULL) {  
printf("No mother  
(already at root)!\n");  
return NULL;  
}
```

```
}
roottmp = t->Mother;
//printf(" %d ",
roottmp->data);
return roottmp;
}

void
freeTree(TYPE_NODEPTR
node) {
if (node != NULL) {
freeTree(node->left);
freeTree(node->right);
free(node);
}
}

void
preOrderTraversal(TYPE_N
ODEPTR node) {
if (node != NULL) {
printf(" %d ", node-
>data);
preOrderTraversal(node-
>left);
preOrderTraversal(node-
>right);
}
}

void
inorderTraversal(TYPE_NO
DEPTR node) {
```

```
if (node != NULL) {  
    inorderTraversal(node->left);  
    printf(" %d ", node->data);  
    inorderTraversal(node->right);  
}  
}
```

```
void  
postOrderTraversal(TYPE_  
NODEPTR node) {  
    if (node != NULL) {  
        postOrderTraversal(node->left);  
        postOrderTraversal(node->right);  
        printf(" %d ", node->data);  
    }  
}
```

```
void printMenu(void){  
}
```

```
void funcSelect() {  
    int choice = 0;  
    int data = 0;  
    const char *Select[12] =  
    {  
        "Exit The Program (Enter  
        0)",
```

```
"Create Root Node",
"Insert Left Node",
"Insert Right Node",
"Go to Left",
"Go to Right",
"Go to Mother",
"Display Tree (from
current position)",
"Pre-Order Traversal
(whole tree)",
"In-Order Traversal
(whole tree)",
"Post-Order Traversal
(whole tree)",
"option 11;"
};
printf("\n=====
=====
=====\\n");
printf("| SELECT
FUNCTION |\\n");
printf("=====
=====
=====\\n");
for (int i = 0; i < 12;
i++) {
printf(" %2d) %s\\n", i ,
Select[i]);
}
if (roottmp != NULL) {
printf("-----
-----
-----\\n");
```

```
printf(" Current
position: %d\n",
roottmp->data);
}
printf("=====
=====
=====\\n");
printf("Enter your
choice: ");
scanf("%d", &choice);

if (choice >= 2 &&
choice <= 10 && roottmp
== NULL) {
printf("!! Please create
the root node first
(option 1) !!\\n");
return;
}

switch (choice) {
case 0:
printf("Exit the
program!\\n");
freeTree(root);
root = NULL;
roottmp = NULL;
running = 0;
break;

case 1:
printf("Enter data to
create first node: ");
```

```
scanf("%d", &data);
CreateNode(data);
break;

case 2:
printf("Enter data to
insert left of %d: ",
roottmp->data);
scanf("%d", &data);
insert_left(roottmp,
data);
break;

case 3:
printf("Enter data to
insert right of %d: ",
roottmp->data);
scanf("%d", &data);
insert_right(roottmp,
data);
break;

case 4:
printf("Go to Left:\n");
gotoLeft(roottmp);
break;

case 5:
printf("Go to Right:
\n");
gotoRight(roottmp);
break;
```

```
case 6:
printf("Go to Mother:
\n");
gotoMother(roottmp);
break;

case 7:
printf("Display Tree
(in-order, from current
position):\n");
Displaytree(roottmp);
printf("\n");
break;

case 8:
printf("Pre-Order
Traversal:\n");
preOrderTraversal(roottm
p);
printf("\n");
break;

case 9:
printf("In-Order
Traversal:\n");
inorderTraversal(roottmp
);
printf("\n");
break;

case 10:
printf("Post-Order
Traversal:\n");
```

```
postOrderTraversal(roottmp);
printf("\n");
break;
case 11:
printf("-----7---
-----\n");
inorderTraversal(root);
printf("\n");
insert_left(roottmp,4);
inorderTraversal(root);
printf("\n");
insert_right(roottmp,12);
;
inorderTraversal(root);
printf("\n");

roottmp =
gotoLeft(roottmp);
insert_left(roottmp,2);
inorderTraversal(root);
printf("\n");

insert_right(roottmp,6);
inorderTraversal(root);
printf("\n");

roottmp =
gotoMother(roottmp);
roottmp =
gotoRight(roottmp);
insert_left(roottmp,10);
inorderTraversal(root);
```

```
printf("\n");
insert_right(roottmp, 14)
;
inorderTraversal(root);
printf("\n");
break;
```

```
default:
printf("-----
-----
\n");
printf("| Invalid
choice! Please select
again! |\n");
printf("-----
-----
\n");
}
}
```

```
int main(){
while (running) {
funcSelect();
}
return 0;
}
```